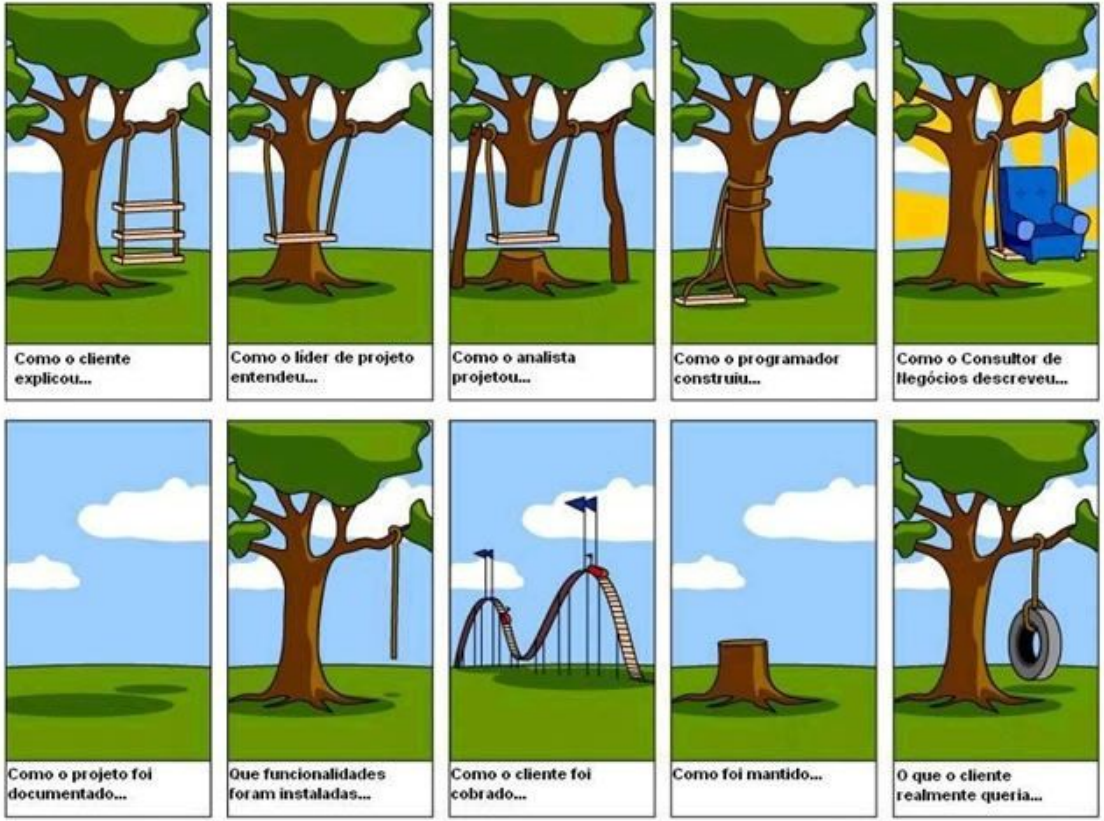
Engenharia e Processos de Software

Prof. Manoel Macedo

Mais sobre Requisitos!

1. Requisitos de software são as necessidades e expectativas que um sistema deve atender para cumprir seus objetivos. Eles podem envolver tanto funcionalidades específicas (como permitir o cadastro de usuários) quanto restrições (como atender a normas de segurança). Sem requisitos bem definidos, o software corre o risco de não resolver o problema real do cliente.

2. O processo de **gerenciamento de requisitos** garante que essas necessidades sejam identificadas, documentadas, acompanhadas e adaptadas ao longo do ciclo de vida do software. Ele é essencial para evitar falhas de comunicação entre clientes, usuários e desenvolvedores, prevenindo erros que podem comprometer todo o projeto.



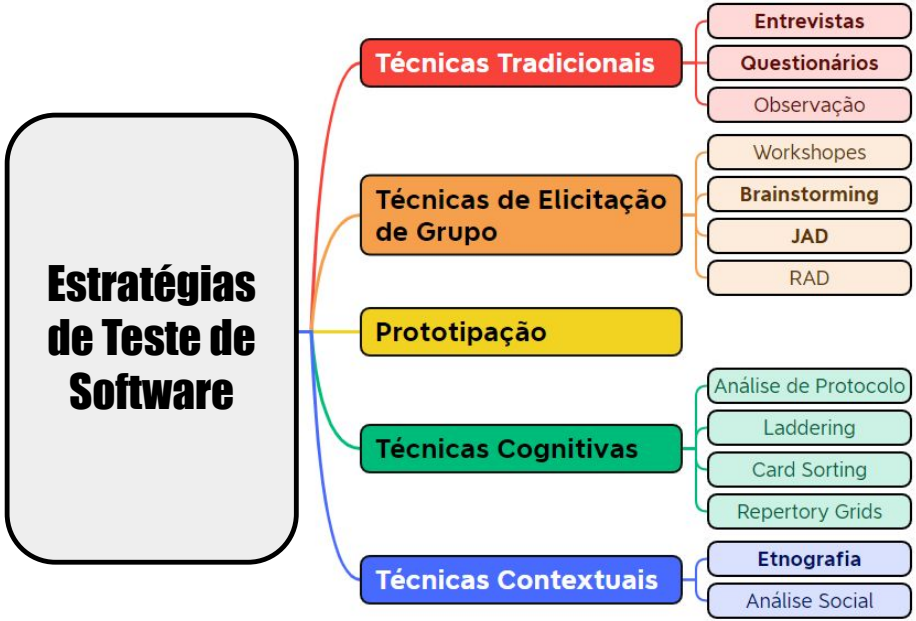
Mais sobre Requisitos!

3. Uma prática central no gerenciamento de requisitos é a **rastreabilidade**, que permite acompanhar cada requisito desde a sua origem até a implementação e os testes. Dessa forma, é possível verificar se todos os objetivos foram atendidos e controlar o impacto de mudanças durante o desenvolvimento.

4. A **elicitação de requisitos** é o processo de descobrir e coletar informações sobre o que o sistema deve fazer. Para isso, são usadas técnicas como entrevistas, questionários, observação de processos e análise de documentos. O objetivo é compreender tanto as necessidades explícitas dos usuários quanto os aspectos implícitos do negócio.

Mais sobre Requisitos!

5. As **entrevistas** são uma das técnicas mais usadas de elicitação, permitindo interação direta entre analistas e stakeholders. Já os **questionários** são úteis quando há muitos usuários ou quando estão dispersos geograficamente. A **observação** ajuda a capturar detalhes que não são ditos, mas que aparecem no uso real de processos ou sistemas.

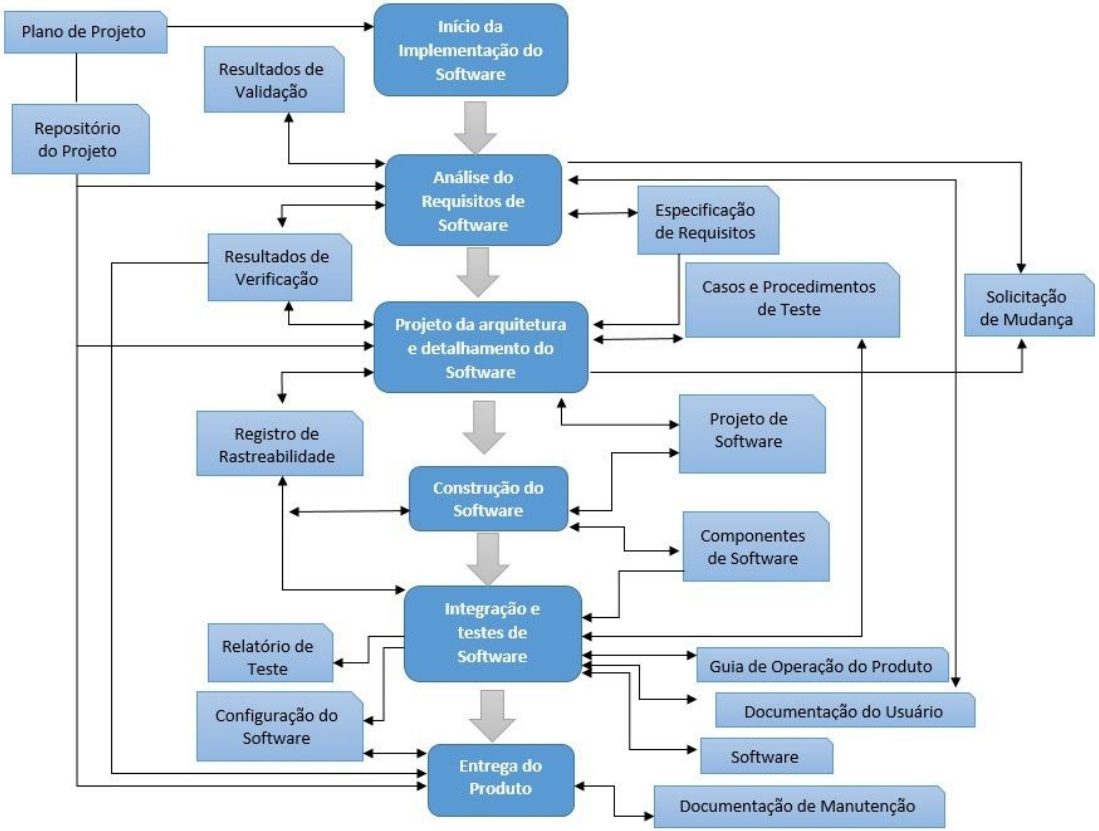


Mais sobre Requisitos!

6. Após coletar as informações, é necessário **organizar os requisitos** em categorias:

- **Requisitos de processo:** como o produto será desenvolvido.
- **Requisitos de projeto:** restrições de tempo, custo ou recursos.
- **Requisitos de produto:** funcionalidades e características a serem implementadas no software.

Essa classificação ajuda a equipe a lidar melhor com cada aspecto do desenvolvimento.

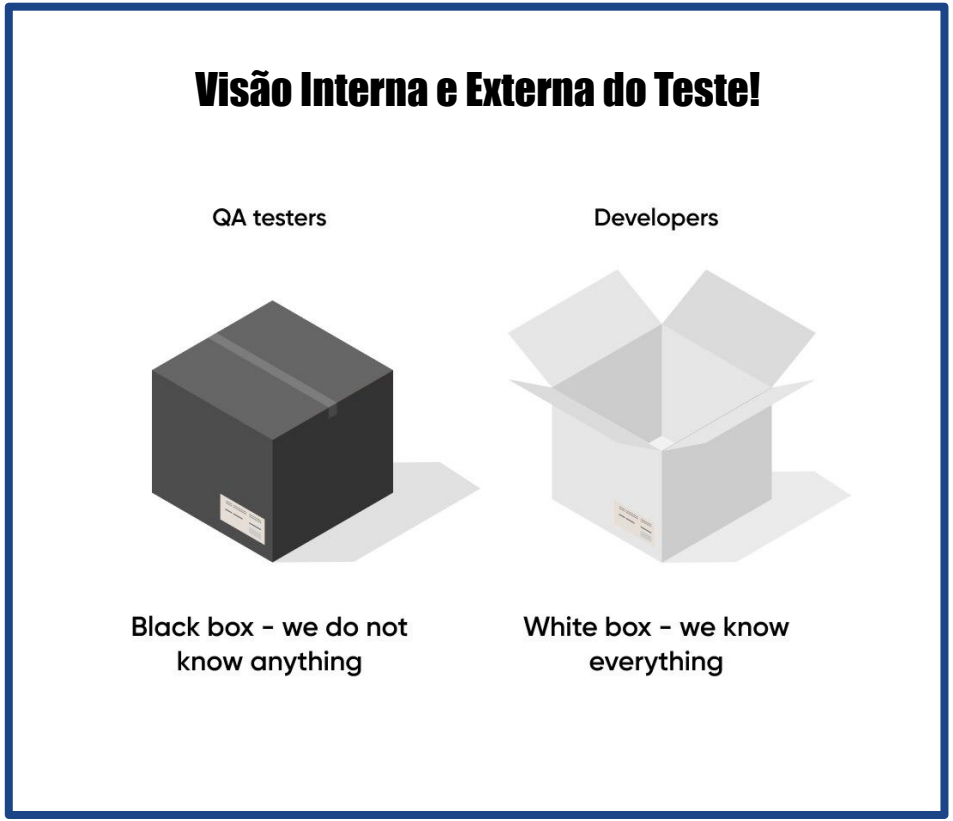


Mais sobre Requisitos!

7. Cada requisito possui **atributos**, como prioridade, estabilidade, impacto na arquitetura e risco. Esses atributos são usados para definir a ordem de implementação, estimar custos e identificar pontos críticos. Por exemplo, um requisito de alta prioridade para o negócio, mas de difícil implementação técnica, exigirá mais planejamento.

8. O ciclo de gerenciamento envolve várias etapas:

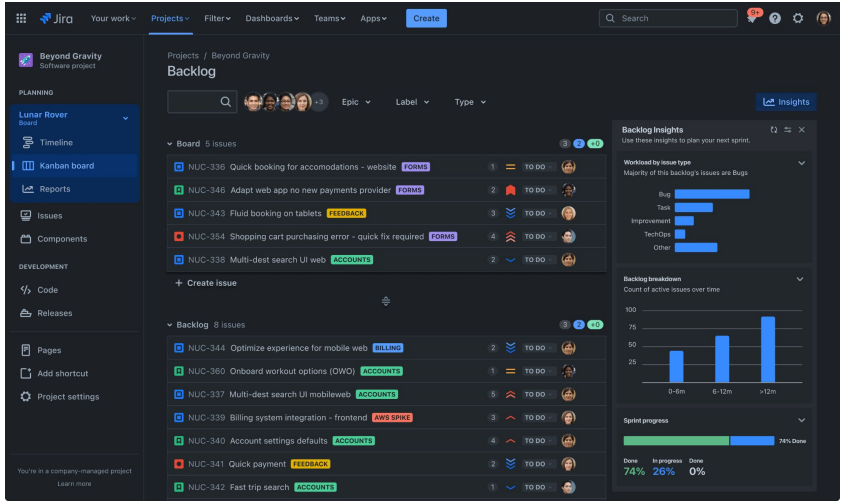
- **Investigação:** entendimento inicial das necessidades.
- **Viabilidade:** análise de custos e benefícios.
- **Projeto:** comparação entre requisitos e design do sistema.
- **Construção e testes:** verificação de que o software atende aos requisitos.
- **Entrega e manutenção:** coleta de feedback para futuras versões.

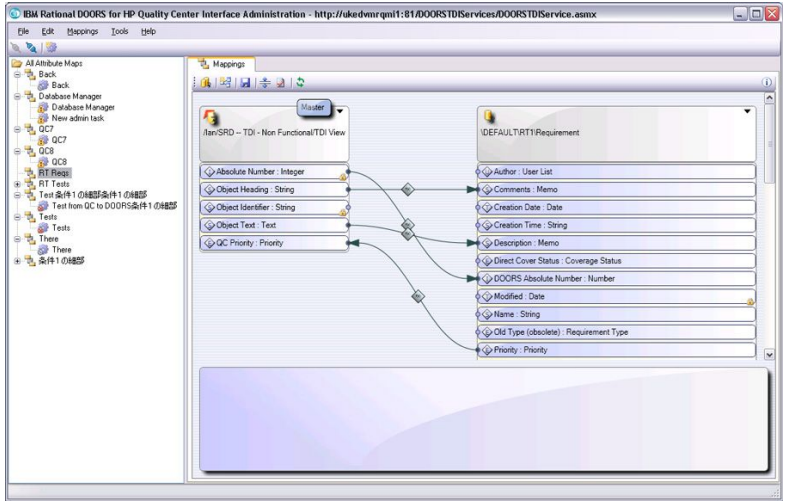


Mais sobre Requisitos!

9. As mudanças em requisitos são inevitáveis. Podem surgir por alterações no mercado, novas leis, erros descobertos ou evolução tecnológica. Por isso, o gerenciamento de mudanças é parte integrante da disciplina, incluindo registro, análise de impacto e decisão sobre implementar ou não cada solicitação.

10. Hoje, existem diversas **ferramentas de apoio** para gerenciar requisitos, como **Jira, IBM Rational DOORS, Jama e Process Street**, que oferecem rastreabilidade automática, versionamento, controle de mudanças e integração com o restante do projeto. Mesmo assim, algumas equipes ainda utilizam planilhas simples, embora com menos recursos.





EXEMPLO - O que é o IBM Rational DOORS?

O **IBM Rational DOORS** é uma ferramenta corporativa voltada para o **gerenciamento de requisitos** em projetos complexos. Seu foco está em oferecer **rastreabilidade completa**, controle de mudanças e colaboração entre equipes distribuídas. Foi projetada especialmente para áreas onde falhas em requisitos podem causar grandes prejuízos, como em **indústria aeroespacial, defesa, automotiva, telecomunicações, saúde e energia**.

Principais Características:

1. **Rastreabilidade ponta a ponta:** permite vincular requisitos de alto nível (ex.: regulatórios) a requisitos técnicos, casos de teste e até resultados de validação.
2. **Controle de mudanças:** qualquer alteração feita em um requisito é registrada, com histórico e justificativa, o que evita perda de informações e facilita auditorias.
3. **Colaboração em equipe:** suporta múltiplos usuários trabalhando simultaneamente, com permissões de acesso e versionamento.
4. **Integração com outras ferramentas:** pode se integrar ao **IBM Engineering Test Management** e a sistemas de ALM (*Application Lifecycle Management*), criando uma cadeia de controle que conecta requisitos → design → código → testes.
5. **Conformidade regulatória:** ajuda empresas a atender normas como **ISO 26262 (automotiva), DO-178C (aeronáutica)** ou exigências da **FDA (saúde)**.

Por que é útil no trabalho com requisitos?

- Em projetos de **grande porte**, lidar com centenas ou milhares de requisitos é inviável em planilhas ou documentos. O DOORS organiza tudo em um banco de dados estruturado, com visualização hierárquica e links de dependência.
- Reduz riscos de falhas de comunicação entre times, já que mantém um **repositório centralizado**.
- Facilita auditorias e inspeções externas, mostrando **quem alterou o quê, quando e por quê**.
- Permite ao gerente de projetos **analisar impactos**: se um requisito muda, a ferramenta mostra quais outros serão afetados (funcionalidades, arquitetura, testes).

Em resumo: o **IBM Rational DOORS é como um "ERP dos requisitos"**, garantindo que nada se perca e que o software esteja sempre alinhado às necessidades do negócio, às normas e aos usuários.

Curiosidades úteis para o profissional de TI

11. Em métodos ágeis, os requisitos costumam ser descritos como **user stories** (“histórias de usuário”), que são curtas descrições do que o usuário espera do sistema. Elas são simples, negociáveis e fáceis de priorizar, tornando o processo mais adaptável às mudanças. Esse formato ajuda a reduzir a burocracia sem perder a clareza.

12. Um dado curioso é que estudos apontam que **mais de 40% das falhas em projetos de software estão relacionadas a problemas em requisitos**, seja por má comunicação, documentação incompleta ou mudanças não controladas. Ou seja, dominar essa área pode aumentar significativamente a taxa de sucesso dos projetos de TI.

Exemplo prático

Imagine que uma escola solicite um sistema para **matrícula online**. O cliente pede que o aluno possa escolher disciplinas, gerar boleto e receber confirmação por e-mail. Esses são **requisitos funcionais**. Além disso, o sistema deve ser capaz de suportar **500 acessos simultâneos** (requisito não funcional). Se esses requisitos não forem bem documentados, pode acontecer de a matrícula funcionar, mas travar quando muitos alunos acessam ao mesmo tempo, frustrando a instituição e os usuários.

Avançando nos conceitos!

Em ambientes corporativos complexos, os requisitos de software não podem ser tratados de forma isolada. É essencial considerar **atributos de requisitos**, como **estabilidade, rastreabilidade, impacto arquitetural, risco e prioridade**. Requisitos instáveis ou mal definidos tendem a gerar retrabalho e custos elevados, especialmente quando já se avançou para fases de codificação e testes. Além disso, em sistemas distribuídos ou críticos (como financeiros e de saúde), os requisitos precisam ser constantemente avaliados quanto ao **impacto em desempenho, segurança, interoperabilidade e conformidade regulatória**, exigindo técnicas mais sofisticadas de análise e validação.

Outro ponto crucial para profissionais de TI é a escolha da abordagem de **documentação e gestão de requisitos**. Enquanto métodos tradicionais utilizam documentos extensos de Especificação de Requisitos de Software (ERS), metodologias ágeis privilegiam **artefatos leves**, como *user stories* e *product backlog items*, apoiados em ferramentas como Jira, Trello ou Azure DevOps. Para contextos regulados ou de missão crítica, ainda se faz necessária a rastreabilidade completa com apoio de soluções como **IBM Rational DOORS ou Jama**, permitindo ligação entre requisitos, design, testes e incidentes. O equilíbrio entre **formalismo e agilidade** é o que garante eficiência sem comprometer a confiabilidade do software.

Comparação – Requisitos em Métodos Ágeis x Tradicionais

Aspecto	Métodos Tradicionais (ex.: Cascata, RUP)	Métodos Ágeis (ex.: Scrum, XP, SAFe)
Forma de documentação	Extensos documentos de **Especificação de Requisitos de Software (ERS)**.	Artefatos leves como **User Stories**, Backlog e Acceptance Criteria.
Momento da definição	Grande parte dos requisitos é definida **antes da implementação**.	Requisitos são descobertos e refinados **ao longo do desenvolvimento**.
Flexibilidade a mudanças	Mudanças são **custosas** e requerem revisões formais.	Mudanças são esperadas e **incorporadas rapidamente** em novas sprints.
Rastreabilidade	Uso de ferramentas formais (ex.: **IBM DOORS**, documentação em nível detalhado).	Uso de quadros e ferramentas ágeis (ex.: **Jira, Trello**) com foco em visibilidade do backlog.
Validação	Testes e validações ocorrem após longos ciclos de desenvolvimento.	Validação contínua com **feedback rápido do cliente** a cada entrega incremental.
Envolvimento do cliente	Cliente participa no início e no fim (aprovação final).	Cliente participa **ativamente durante todo o processo**.
Foco principal	**Conformidade** com especificações formais.	**Valor entregue ao usuário** de forma incremental.

Uma Questão!

No processo de Engenharia de Requisitos, a rastreabilidade é importante porque:

a) Permite reduzir o tempo de codificação.

b) Garante que todas as linhas de código sejam testadas.

c) Auxilia a acompanhar a evolução de cada requisito e seus impactos no sistema.

d) Elimina a necessidade de documentar os requisitos.

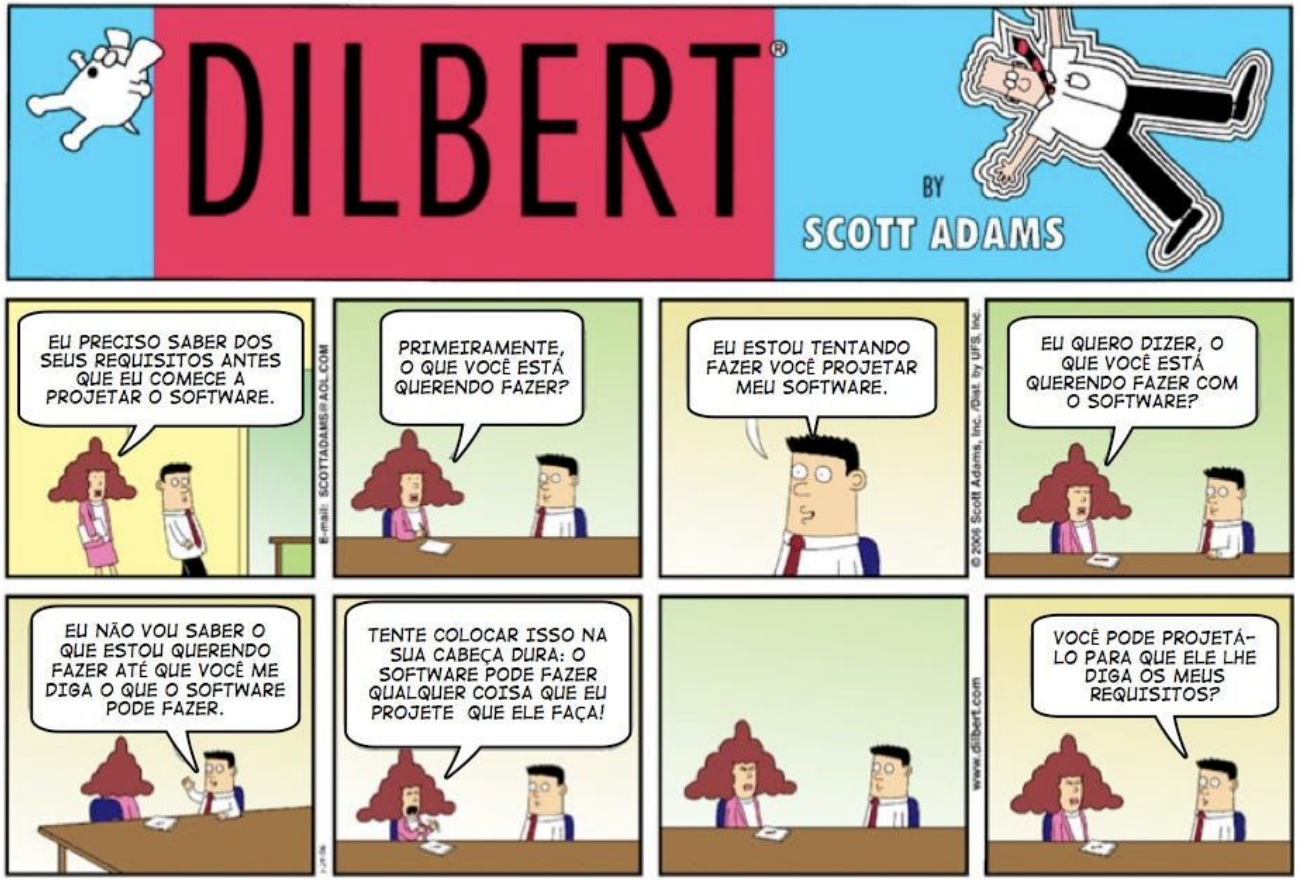
e) Substitui a etapa de testes de aceitação.

RESUMINDO…

Na computação, **requisitos de software** representam as funcionalidades, restrições e qualidades que um sistema deve possuir para atender às necessidades de usuários e organizações. Eles se dividem em **funcionais** (o que o sistema deve fazer, como cadastrar usuários ou processar pagamentos) e **não funcionais** (como desempenho, segurança, usabilidade e confiabilidade).

O processo de **engenharia de requisitos** envolve atividades como **elicitação**, **análise**, **especificação**, **validação** e **gerenciamento**. Na elicitação, técnicas como **entrevistas, questionários, brainstorming, prototipagem e observação** ajudam a identificar expectativas e problemas reais. A análise busca organizar, priorizar e detectar conflitos entre os requisitos. A especificação registra de forma clara, seja em documentos formais, seja em **user stories** em métodos ágeis. A validação garante que o que foi definido realmente atende às necessidades, e o gerenciamento lida com mudanças inevitáveis ao longo do ciclo de vida do software.

Em suma, requisitos funcionam como a **ponte entre o problema e a solução computacional**. A qualidade do software depende diretamente da qualidade com que esses requisitos são coletados, compreendidos e mantidos. Dominar técnicas e boas práticas de requisitos é essencial para qualquer profissional de TI que deseje entregar sistemas úteis, confiáveis e sustentáveis.



© Scott Adams, Inc./Dist. by UFS, Inc.

Obrigado!

[manoelmacedo@yahoo.com]

Fontes:

Normas ISO - POHL, K.; RUPP, C. (Requirements engineering fundamentals: a study guide for the certified professional for requirements engineering exam, foundation level) - SCHWABER, K.; SUTHERLAND, J. (Guia do SCRUM: um guia definitivo para o SCRUM: as regras do jogo.) - Amazon - SOMMERVILLE, I.; https://acertbr.com.br/engenharia-de-requisitos-conheca-as-etapas-do-processo/ - SAWYER, P. (Requirements engineering: a good practice guide) - Wikipédia - Pinterest - USP - Cybermedian - www.estrategiaconcursos.com.br - https://imasters.com.br/devsecops/utilizacao-de-checklist-para-validacao-de-requisitos-de-software - Sheila Reinehr (SAGAH) - https://conteudo.catolica.edu.br/conteudos/nbt_cursos/engenharia_requisitos/tema_01/ - WIEGERS, K. E.; BEATTY, J. (Software requirements) - MIT - www.inf.ufsc.br - UNIT - Revista Superinteressante - - Medium.com - Manifesto Ágil - Google Imagens…